

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 03

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	30%	20%	15%	20%	15%	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Total Marks :

Annexure A

Short Answer Test

1. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.
2. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.
3. Write a Python program to implement the **Support Vector Machine (SVM)** algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Naïve Bayes Classifier

```
# Naive Bayes Classifier
```

```
# With and Without Laplace Smoothing
```

```
from collections import Counter, defaultdict
```

```
# Dataset
```

```
# Features: Outlook, Temperature, Humidity
```

```
# Class: Play
```

```
data = [  
    ["Sunny", "Hot", "High", "No"],  
    ["Sunny", "Hot", "High", "No"],  
    ["Overcast", "Hot", "High", "Yes"],  
    ["Rain", "Mild", "High", "Yes"],  
    ["Rain", "Cool", "Normal", "Yes"],  
    ["Rain", "Cool", "Normal", "No"],  
    ["Overcast", "Cool", "Normal", "Yes"],  
    ["Sunny", "Mild", "High", "No"],  
    ["Sunny", "Cool", "Normal", "Yes"],  
    ["Rain", "Mild", "Normal", "Yes"],  
    ["Sunny", "Mild", "Normal", "Yes"],  
    ["Overcast", "Mild", "High", "Yes"],  
    ["Overcast", "Hot", "Normal", "Yes"],  
    ["Rain", "Mild", "High", "No"]  
]
```

```
# Test data
```

```
test_data = [  
    ["Sunny", "Cool", "High", "No"],  
    ["Overcast", "Mild", "High", "Yes"],  
    ["Rain", "Cool", "Normal", "Yes"],  
    ["Sunny", "Mild", "Normal", "Yes"]  
]
```

```
def train_naive_bayes(data, smoothing=False):  
    n = len(data)  
    class_counts = Counter(row[-1] for row in data)  
    classes = list(class_counts.keys())
```

```

# Prior probabilities
priors = {
    c: class_counts[c] / n
    for c in classes
}

# Conditional probabilities
cond_probs = defaultdict(dict)

num_features = len(data[0]) - 1

for feature_index in range(num_features):
    values = set(row[feature_index] for row in data)
    num_values = len(values)

    for c in classes:
        class_rows = [row for row in data if row[-1] == c]
        class_count = len(class_rows)

        for value in values:
            count = sum(
                row[feature_index] == value
                for row in class_rows
            )

            if smoothing:
                # Laplace smoothing
                probability = (count + 1) / (
                    class_count + num_values
                )
            else:
                probability = count / class_count

            cond_probs[(feature_index, value)][c] = probability

return priors, cond_probs

def predict(instance, priors, cond_probs):

```

```

scores = {}

for c in priors:
    probability = priors[c]

    for feature_index, value in enumerate(instance):
        probability *= cond_probs[
            (feature_index, value)
        ].get(c, 0)

    scores[c] = probability

# If all probabilities become zero
if all(value == 0 for value in scores.values()):
    return max(priors, key=priors.get)

return max(scores, key=scores.get)

def evaluate(test_data, smoothing=False):
    priors, cond_probs = train_naive_bayes(
        data, smoothing
    )

    correct = 0
    predictions = []

    for row in test_data:
        instance = row[:-1]
        actual = row[-1]

        predicted = predict(
            instance,
            priors,
            cond_probs
        )

        predictions.append(predicted)

        if predicted == actual:

```

```

        correct += 1

    accuracy = (correct / len(test_data)) * 100

    return priors, predictions, accuracy

# -----
# WITHOUT LAPLACE SMOOTHING
# -----

priors, predictions, accuracy = evaluate(
    test_data,
    smoothing=False
)

print("NAIVE BAYES CLASSIFIER")
print("=" * 50)

print("\nPrior Probabilities:")
for c, p in priors.items():
    print(c, "=", round(p, 4))

print("\nWithout Laplace Smoothing:")
for i, row in enumerate(test_data):
    print(
        "Instance:", row[:-1],
        "Actual:", row[-1],
        "Predicted:", predictions[i]
    )

print("Accuracy =", round(accuracy, 2), "%")

# -----
# WITH LAPLACE SMOOTHING
# -----

priors, predictions, accuracy = evaluate(
    test_data,

```

```

    smoothing=True
)

print("\nWith Laplace Smoothing:")
for i, row in enumerate(test_data):
    print(
        "Instance:", row[:-1],
        "Actual:", row[-1],
        "Predicted:", predictions[i]
    )

print("Accuracy =", round(accuracy, 2), "%")

```

output:

NAIVE BAYES CLASSIFIER

Prior Probabilities:

No = 0.3571

Yes = 0.6429

Without Laplace Smoothing:

Instance: ['Sunny', 'Cool', 'High'] Actual: No Predicted: No

Instance: ['Overcast', 'Mild', 'High'] Actual: Yes Predicted: Yes

Instance: ['Rain', 'Cool', 'Normal'] Actual: Yes Predicted: Yes

Instance: ['Sunny', 'Mild', 'Normal'] Actual: Yes Predicted: Yes

Accuracy = 100.0 %

With Laplace Smoothing:

Instance: ['Sunny', 'Cool', 'High'] Actual: No Predicted: No

Instance: ['Overcast', 'Mild', 'High'] Actual: Yes Predicted: Yes

Instance: ['Rain', 'Cool', 'Normal'] Actual: Yes Predicted: Yes

Instance: ['Sunny', 'Mild', 'Normal'] Actual: Yes Predicted: Yes

2. Genetic Algorithm

Genetic Algorithm

Maximize $f(x) = x^2$

```

import random

# Parameters
POPULATION_SIZE = 10
CHROMOSOME_LENGTH = 5
GENERATIONS = 20
MUTATION_RATE = 0.05
CROSSOVER_RATE = 0.8

# Convert binary chromosome to decimal
def binary_to_decimal(chromosome):
    value = 0

    for bit in chromosome:
        value = value * 2 + bit

    return value

# Fitness function
def fitness(chromosome):
    x = binary_to_decimal(chromosome)
    return x * x

# Generate random chromosome
def create_chromosome():
    return [
        random.randint(0, 1)
        for _ in range(CHROMOSOME_LENGTH)
    ]

# Create initial population
def create_population():
    return [
        create_chromosome()
        for _ in range(POPULATION_SIZE)
    ]

```

```
]
```

```
# Selection using tournament selection
```

```
def selection(population):
```

```
    selected = []
```

```
    for _ in range(POPULATION_SIZE):
```

```
        a = random.choice(population)
```

```
        b = random.choice(population)
```

```
        if fitness(a) > fitness(b):
```

```
            selected.append(a[:])
```

```
        else:
```

```
            selected.append(b[:])
```

```
    return selected
```

```
# Single-point crossover
```

```
def crossover(parent1, parent2):
```

```
    if random.random() < CROSSOVER_RATE:
```

```
        point = random.randint(
```

```
            1,
```

```
            CHROMOSOME_LENGTH - 1
```

```
        )
```

```
    child1 = (
```

```
        parent1[:point] +
```

```
        parent2[point:]
```

```
    )
```

```
    child2 = (
```

```
        parent2[:point] +
```

```
        parent1[point:]
```

```
    )
```

```
    return child1, child2
```

```

return parent1[:, parent2[:]]

# Mutation
def mutation(chromosome):

    for i in range(CHROMOSOME_LENGTH):

        if random.random() < MUTATION_RATE:
            chromosome[i] = 1 - chromosome[i]

    return chromosome

# Find best chromosome
def best_solution(population):
    return max(
        population,
        key=fitness
    )

# Genetic Algorithm
population = create_population()

print("GENETIC ALGORITHM")
print("=" * 50)

for generation in range(GENERATIONS):

    # Calculate fitness
    fitness_values = [
        fitness(chromosome)
        for chromosome in population
    ]

    best = best_solution(population)

    print(

```

```
    "Generation:",  
    generation + 1,  
    "Best x:",  
    binary_to_decimal(best),  
    "Fitness:",  
    fitness(best)  
)
```

```
# Selection  
selected = selection(population)
```

```
# Create new population  
new_population = []
```

```
for i in range(0, POPULATION_SIZE, 2):
```

```
    parent1 = selected[i]  
    parent2 = selected[  
        (i + 1) % POPULATION_SIZE  
    ]
```

```
# Crossover  
child1, child2 = crossover(  
    parent1,  
    parent2  
)
```

```
# Mutation  
child1 = mutation(child1)  
child2 = mutation(child2)
```

```
new_population.append(child1)  
new_population.append(child2)
```

```
population = new_population[:POPULATION_SIZE]
```

```
# Final solution  
best = best_solution(population)
```

```
print("\nFINAL RESULT")
print("-" * 50)

print("Best Chromosome:", best)
print(
    "Best Solution x:",
    binary_to_decimal(best)
)
print("Maximum Fitness:", fitness(best))
```

output:

GENETIC ALGORITHM

```
=====
Generation: 1 Best x: 31 Fitness: 961
Generation: 2 Best x: 31 Fitness: 961
Generation: 3 Best x: 31 Fitness: 961
Generation: 4 Best x: 31 Fitness: 961
Generation: 5 Best x: 31 Fitness: 961
Generation: 6 Best x: 31 Fitness: 961
Generation: 7 Best x: 31 Fitness: 961
Generation: 8 Best x: 31 Fitness: 961
Generation: 9 Best x: 31 Fitness: 961
Generation: 10 Best x: 31 Fitness: 961
Generation: 11 Best x: 31 Fitness: 961
Generation: 12 Best x: 31 Fitness: 961
Generation: 13 Best x: 31 Fitness: 961
Generation: 14 Best x: 31 Fitness: 961
Generation: 15 Best x: 31 Fitness: 961
Generation: 16 Best x: 31 Fitness: 961
Generation: 17 Best x: 31 Fitness: 961
Generation: 18 Best x: 31 Fitness: 961
Generation: 19 Best x: 31 Fitness: 961
Generation: 20 Best x: 31 Fitness: 961
```

FINAL RESULT

```
-----
Best Chromosome: [1, 1, 1, 1, 1]
Best Solution x: 31
```

Maximum Fitness: 961

3. Support Vector Machine — Linear Kernel

```
# Linear Support Vector Machine
# Binary Classification
#
# Dataset contains two classes:
# Class 0 and Class 1
```

```
import numpy as np
```

```
# -----
# Training data
# -----
```

```
X_train = np.array([
    [1, 1],
    [2, 1],
    [1, 2],
    [2, 2],
    [6, 6],
    [7, 6],
    [6, 7],
    [7, 7]
], dtype=float)
```

```
y_train = np.array([
    -1, -1, -1, -1,
    1, 1, 1, 1
], dtype=float)
```

```
# -----
# Test data
# -----
```

```
X_test = np.array([
    [1.5, 1.5],
```

```
[2, 1.5],  
[6.5, 6.5],  
[7, 6]  
, dtype=float)
```

```
y_test = np.array([  
    -1,  
    -1,  
     1,  
     1  
, dtype=float)
```

```
# -----  
# Linear SVM using gradient descent  
# -----
```

```
class LinearSVM:
```

```
    def __init__(  
        self,  
        learning_rate=0.001,  
        lambda_param=0.01,  
        epochs=1000  
    ):  
        self.lr = learning_rate  
        self.lambda_param = lambda_param  
        self.epochs = epochs  
        self.w = None  
        self.b = None
```

```
    def fit(self, X, y):
```

```
        samples, features = X.shape
```

```
        self.w = np.zeros(features)  
        self.b = 0
```

```
        for epoch in range(self.epochs):
```

```

for i, x in enumerate(X):

    condition = y[i] * (
        np.dot(x, self.w) + self.b
    )

    # Hinge-loss condition
    if condition >= 1:

        dw = (
            2 * self.lambda_param * self.w
        )

        db = 0

    else:

        dw = (
            2 * self.lambda_param * self.w
            - y[i] * x
        )

        db = -y[i]

    # Update weights
    self.w -= self.lr * dw
    self.b -= self.lr * db

def predict(self, X):

    linear_output = np.dot(X, self.w) + self.b

    return np.sign(linear_output)

# -----
# Train model
# -----

model = LinearSVM(

```

```

        learning_rate=0.001,
        lambda_param=0.01,
        epochs=1000
    )

model.fit(X_train, y_train)

# -----
# Prediction
# -----

predictions = model.predict(X_test)

print("LINEAR SUPPORT VECTOR MACHINE")
print("=" * 50)

print("\nWeights:")
print(model.w)

print("\nBias:")
print(model.b)

print("\nPredictions:")

for i in range(len(X_test)):

    print(
        "Input:",
        X_test[i],
        "Actual:",
        int(y_test[i]),
        "Predicted:",
        int(predictions[i])
    )

# -----
# Accuracy
# -----

```

```

accuracy = np.mean(
    predictions == y_test
) * 100

print("\nAccuracy:", round(accuracy, 2), "%")

```

```

# -----
# Decision boundary
# -----

```

```

print("\nDecision Function:")
print(
    "f(x) = w1*x1 + w2*x2 + b"
)

```

```

print(
    "f(x) =",
    round(model.w[0], 4),
    "* x1 +",
    round(model.w[1], 4),
    "* x2 +",
    round(model.b, 4)
)

```

Output;

LINEAR SUPPORT VECTOR MACHINE

=====

Weights:

[0.24861984 0.24970705]

Bias:

-1.93399999999998978

Predictions:

Input: [1.5 1.5] Actual: -1 Predicted: -1

Input: [2. 1.5] Actual: -1 Predicted: -1

Input: [6.5 6.5] Actual: 1 Predicted: 1

Input: [7. 6.] Actual: 1 Predicted: 1

Accuracy: 100.0 %

Decision Function:

$$f(x) = w_1 * x_1 + w_2 * x_2 + b$$

$$f(x) = 0.2486 * x_1 + 0.2497 * x_2 + -1.934$$